Web Development — Day 1

Hero Section & CSS Positioning Revision Notes

This document collects the concepts and doubts covered during the first day while building the University Landing Page hero section.

1. Basic Project Structure

The project was organised like this:

```
Project1/
■
■■■ index.html
■■■ style.css
■■■ images/
    ■■■ University-image.jpg
```

index.html contains the structure and content of the webpage. **style.css** controls the design and layout. Images are stored inside the **images** folder.

2. Linking HTML and CSS

```
<link rel="stylesheet" href="style.css">
```

This line connects the HTML file with the CSS file. Without the correct link, CSS changes will not appear.

3. Background Image in the Hero Section

```
.header{
    min-height: 100vh;
    width: 100%;
    background-image: url("images/University-image.jpg");
    background-size: cover;
    background-position: center;
}
```

background-image places an image behind the content of an element.

background-size: cover makes the image cover the entire container. Some parts of the image may be cropped.

background-position: center keeps the central part of the image visible.

Why min-height: 100vh was important

100vh means 100% of the visible browser height. Therefore, **min-height: 100vh** makes the hero section at least as tall as the screen. When it was removed, the background image appeared to take only the height required by the content, which is why it looked much smaller.

4. Resetting Default Margin and Padding

```
*{
    margin: 0;
    padding: 0;
}
```

Browsers add some default spacing to elements. This reset removes that default spacing so that the layout can be controlled manually.

5. Padding Percentages — What Do 2% and 6% Mean?

```
padding: 2% 6%;
```

This is shorthand:

• **2%** is used for top and bottom padding.
• **6%** is used for left and right padding.

A percentage does not mean a fixed number of pixels. It changes according to the relevant size of the containing element. For example, if the relevant width is 1000px, 6% is 60px. Therefore, percentage-based spacing is responsive.

6. Flexbox Navigation

```
nav{
    display: flex;
    padding: 2% 6%;
    justify-content: space-between;
    align-items: center;
}
```

display: flex places the direct child elements in a flexible layout.

align-items: center aligns flex items along the cross axis, which is usually vertically for a normal row layout.

space-between vs space-evenly

justify-content: space-between: the first item goes to the starting edge and the last item goes to the ending edge. The remaining items have equal space between them.

justify-content: space-evenly: equal space is placed everywhere, including before the first item, between items, and after the last item.

Simple example:

If there are three boxes:

• **space-between**: Box 1 touches the left side, Box 3 touches the right side, and equal gaps appear between the boxes.
• **space-evenly**: equal gaps appear on both outer sides as well as between every box.

7. Navigation Links

```
.navlinks ul{
    display: flex;
    list-style: none;
}

.navlinks ul li{
    margin-left: 20px;
}

.navlinks ul li a{
    text-decoration: none;
    color: black;
}
```

list-style: none removes the bullet points from the list.

margin-left: 20px creates space between navigation items.

text-decoration: none removes the default underline from links.

8. CSS Positioning — The Main Concept

The most important positioning concept learned was the relationship between a **parent** and a **child**.

```
.header{
    position: relative;
}

.text-box{
    position: absolute;
}
```

When a child has **position: absolute**, it is positioned relative to its nearest ancestor that has a position other than the default **static**. In this project, the parent **.header** is set to **position: relative**, so the absolutely positioned **.text-box** uses the header as its positioning reference.

Important revision point: To position a child absolutely with respect to a specific parent, a common pattern is to set the parent to **position: relative** and the child to **position: absolute**.

If the parent position is not declared

If the intended parent is not positioned, the absolutely positioned child may use another positioned ancestor or the initial containing block instead. Therefore, if the child must be positioned relative to a particular parent, set that parent to **position: relative**.

9. position: absolute — Margin, Padding and Border Confusion

position: absolute does not mean the child starts from the parent's margin. The positioning offsets such as **top** and **left** are calculated from the positioning reference established by the positioned ancestor.

In simple terms: make the parent the reference area using **position: relative**, then use **top**, **left**, **right**, or **bottom** to move the absolutely positioned child within that reference.

10. What top: 50% and left: 50% Actually Do

```
position: absolute;
top: 50%;
left: 50%;
```

These properties move the **top-left corner of the child box** to the center point of the parent:

• **top: 50%** → the child's top edge is placed 50% down from the parent's positioning area.
• **left: 50%** → the child's left edge is placed 50% across from the parent's positioning area.

Very important: The child's center is not automatically placed at the center. Only its **top-left corner** reaches the center point.

Example

Imagine a parent box of 1000px width and 600px height.

At **left: 50%**, the child's left edge moves to 500px from the left.

At **top: 50%**, the child's top edge moves to 300px from the top.

If nothing else is done, the entire child box continues to the right and downward from that point. This is why the text initially looked too far to the right and lower than expected.

11. Why transform: translate(-50%, -50%) Is Needed

```
transform: translate(-50%, -50%);
```

After **top: 50%** and **left: 50%**, the child's top-left corner is at the parent's center. The transform shifts the child back:

• **translateX(-50%)** moves the child left by 50% of the **child's own width**.
• **translateY(-50%)** moves the child upward by 50% of the **child's own height**.

Therefore, the child's own center moves onto the parent's center.

The difference between top/left and transform

top: 50%; left: 50% use the parent/reference area to decide where the child's top-left corner should go.

transform: translate(-50%, -50%) uses the child's own size to shift the child back.

Together they create true centering.

Simple memory trick:

top/left → go to the parent's center.
transform → pull the child back by half of itself.

12. Why the Text Looked Far Right Before transform

Without the transform, the left edge of the text box begins at the center of the parent. Since the box has a large width, most of the box extends to the right. That is why the content looked shifted to the right. Adding the negative translate moves the whole box back to center it.

13. text-align: center / left

```
text-align: center;
text-align: left;
```

text-align controls the horizontal alignment of inline content such as text inside an element.

It does **not** change the position of the child box itself.

For example, if **.text-box** is absolutely centered inside **.header**, then **text-align: center** centers the text **inside the text-box**. The position of the text-box relative to the parent is controlled separately by position, top, left and transform.

14. Complete Hero Text Positioning

```
.text-box{
    width: 90%;
    color: black;

    position: absolute;
    top: 50%;
    left: 50%;

    transform: translate(-50%, -50%);
    text-align: left;
}
```

width: 90% gives the text box most of the available width.

position: absolute removes it from the normal document flow and allows precise positioning.

top: 50%; left: 50% place the top-left corner at the center reference point.

transform: translate(-50%, -50%) moves the box back by half of its own width and height.

text-align: left aligns the text inside that box to the left.

15. Styling the Hero Text

```
.text-box h1{
    font-size: 50px;
    color: white;
}

.text-box p{
    color: white;
    margin: 0px 0px 10px 0px;
```

```
}
```

The heading and paragraph are styled separately because different elements may require different font sizes, colours and spacing.

The paragraph margin shown above means: top 0px, right 0px, bottom 10px, left 0px.

16. Images in HTML vs Images in CSS

Images can be added in two main ways.

A. HTML image

```
<img src="images/students.jpg" alt="Students">
```

Use an HTML **img** when the image is actual content of the webpage and has meaning or information.

B. CSS background image

```
background-image: url("images/University-image.jpg");
```

Use a CSS background image mainly when the image is part of the design, decoration or background of a section. The hero background is a good example because text is displayed on top of it.

17. What alt Does

```
alt="Students"
```

The **alt** attribute provides alternative text for an image.

It helps screen readers describe the image and can provide useful text when an image cannot load.

Example: **alt="Students studying in a classroom"** gives meaningful information about the image.

18. margin: auto — Why the Layout Looked Wrong

margin: auto should not be applied randomly to every element.

It is commonly used to horizontally center a block element when the element has an available width smaller than its container, for example:

```
.about-row{
    width: 80%;
    margin: auto;
}
```

This means the left and right automatic margins share the remaining space, helping center the container.

If **margin: auto** is applied to an image or another element without understanding its width and layout context, the layout may behave differently from what was expected.

Important: Do not replace **padding** with **margin: auto**. Padding creates space inside an element; margin creates space outside an element.

19. Quick Difference: Margin vs Padding

Padding = space inside the element, between the content and its border.

Margin = space outside the element, between that element and surrounding elements.

20. Final Hero Section Code

The following represents the main hero section structure studied on Day 1.

```
HTML
```

```
<section class="header">
    <nav>
        <h2>MY UNIVERSITY</h2>

        <div class="navlinks">
            <ul>
                <li><a href="#">HOME</a></li>
                <li><a href="#">ABOUT</a></li>
                <li><a href="#">COURSES</a></li>
                <li><a href="#">CONTACT</a></li>
            </ul>
        </div>
    </nav>

    <div class="text-box">
        <h1>Learn. Grow. Build your future.</h1>
        <p>Welcome to our University, where students learn, explore and build their future.</p>
        <a href="#" class="hero-btn">Visit Us To Know More</a>
    </div>
</section>
```

CSS

```
*{
    margin: 0;
    padding: 0;
}

.header{
    min-height: 100vh;
    width: 100%;
    background-image: url("images/University-image.jpg");
    background-size: cover;
    background-position: center;
    position: relative;
}

nav{
    display: flex;
    padding: 2% 6%;
    justify-content: space-between;
    align-items: center;
}

.navlinks ul{
    display: flex;
    list-style: none;
}

.navlinks ul li{
    margin-left: 20px;
}

.navlinks ul li a{
    text-decoration: none;
    color: black;
}

.text-box{
    width: 90%;
    color: black;
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);
    text-align: left;
}

.text-box h1{
    font-size: 50px;
```

```
    color: white;
}

.text-box p{
    color: white;
    margin: 0px 0px 10px 0px;
}

.hero-btn{
    text-decoration: none;
    color: white;
    border: 1px black solid;
}
```

21. Most Important Revision Points

1. **min-height: 100vh** makes the hero section at least as tall as the visible screen.

2. **background-size: cover** makes the background cover the container.

3. **space-between** pushes the first and last flex items to opposite ends; **space-evenly** creates equal space everywhere.

4. To position a child relative to a chosen parent, commonly use **parent: position: relative** and **child: position: absolute**.

5. **top: 50%; left: 50%** move the child's **top-left corner** to the center reference point.

6. **transform: translate(-50%, -50%)** moves the child back by half of its own width and height.

7. **top/left** use the positioning reference; **transform percentages** use the transformed element's own size.

8. **text-align** aligns text inside an element; it does not position the element itself.

9. Use HTML **img** for meaningful content images and CSS backgrounds for design/background images.

10. **alt** provides alternative text and improves accessibility.

11. **margin** is outside space; **padding** is inside space.

12. Use **margin: auto** deliberately, usually on a container with an appropriate width when centering is needed.

End of Day 1 Revision Notes

Focus especially on the parent–child positioning relationship, the difference between top/left and transform, and the difference between positioning a box versus aligning the text inside that box. These are the key concepts from the hero section.